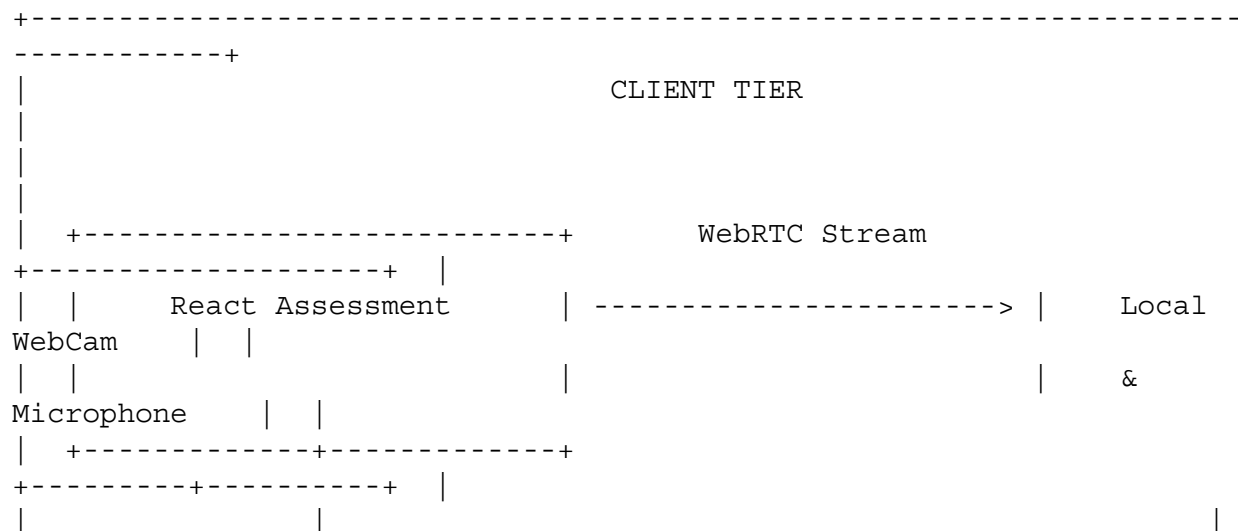
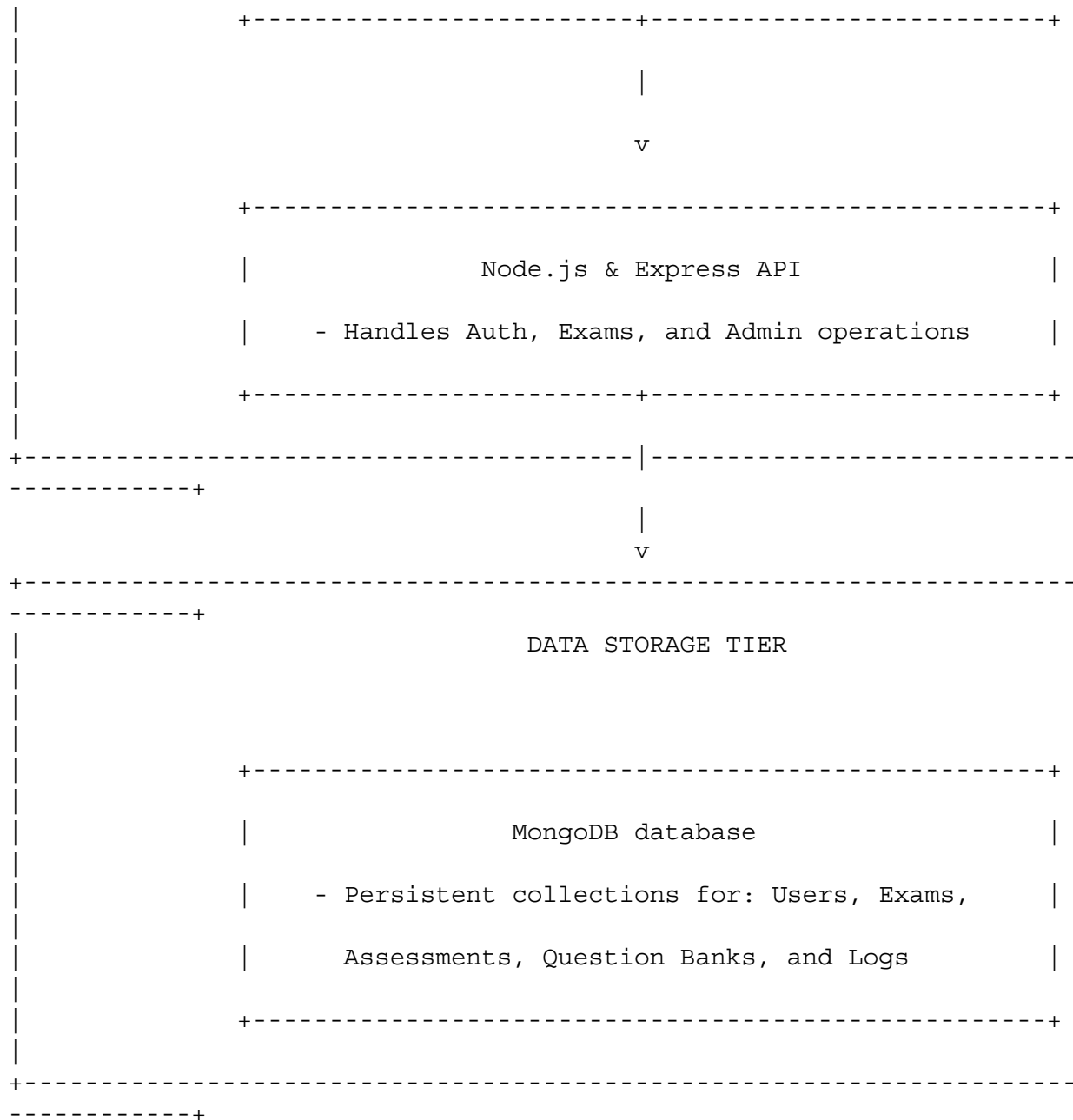






3. Database Schema Design (MongoDB)

Below are the strictly structured schema blueprints for a robust implementation using Mongoose.

```
// 1. User Schema (Students and Administrators)
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  passwordHash: { type: String, required: true },
```

```
    role: { type: String, enum: ['student', 'admin'], default: 'student'
  },
  createdAt: { type: Date, default: Date.now }
});
```

```
// 2. Exam Schema (Question Bank & Rule Settings)
const ExamSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String },
  durationMinutes: { type: Number, required: true },
  totalQuestions: { type: Number, default: 36 },
  maxWarnings: { type: Number, default: 5 },
  rules: {
    enforceFullscreen: { type: Boolean, default: true },
    disableCopyPaste: { type: Boolean, default: true },
    trackGaze: { type: Boolean, default: true },
    trackHeadAngle: { type: Boolean, default: true },
    detectMultipleFaces: { type: Boolean, default: true }
  },
  questions: [{
    questionText: { type: String, required: true },
    options: [{ type: String, required: true }],
    correctOptionIndex: { type: Number, required: true },
    marks: { type: Number, default: 1 }
  }]
});
```

```
// 3. StudentExamSession Schema (Progress tracker & live scoring)
const StudentExamSessionSchema = new mongoose.Schema({
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
    required: true },
  examId: { type: mongoose.Schema.Types.ObjectId, ref: 'Exam',
    required: true },
  status: { type: String, enum: ['active', 'paused', 'submitted',
    'terminated'], default: 'active' },
  startedAt: { type: Date, default: Date.now },
  completedAt: { type: Date },
  answers: [{
    questionId: { type: mongoose.Schema.Types.ObjectId, required: true
  },
    selectedOptionIndex: { type: Number },
    savedAt: { type: Date, default: Date.now }
  }],
  warningCount: { type: Number, default: 0 },
  cumulativeRiskScore: { type: Number, default: 0 },
  ipAddress: { type: String }
});
```

```
// 4. ProctorLog Schema (Historical logging and admin investigation
audit logs)
const ProctorLogSchema = new mongoose.Schema({
  sessionId: { type: mongoose.Schema.Types.ObjectId, ref:
'StudentExamSession', required: true },
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User',
required: true },
  examId: { type: mongoose.Schema.Types.ObjectId, ref: 'Exam',
required: true },
  eventType: {
    type: String,
    enum: [
      'TAB_SWITCHED',
      'FULLSCREEN_EXIT',
      'NO_FACE_DETECTED',
      'MULTIPLE_FACES_DETECTED',
      'LOOKING_AWAY_LEFT',
      'LOOKING_AWAY_RIGHT',
      'SUSPICIOUS_AUDIO',
      'COPY_PASTE_ATTEMPT'
    ],
    required: true
  },
  severity: { type: String, enum: ['low', 'medium', 'high',
'critical'], required: true },
  timestamp: { type: Date, default: Date.now },
  additionalMetadata: { type: mongoose.Schema.Types.Mixed } // Stores
geometric coordinates or debug info
});
```

4. Frontend Component & Folder Architecture

```
frontend/
├── public/
│   ├── models/                # MediaPipe models hosted locally
│   └── for performance
│       ├── face_landmarker.task
│       ├── face_detector.tflite
│       └── favicon.ico
├── src/
│   ├── assets/                # Static images, UI design vectors
│   ├── components/
│   │   ├── Common/
│   │   │   ├── Button.jsx
│   │   │   ├── Modal.jsx
│   │   └── Navigation.jsx
```

```

|   |   |─ Assessment/
|   |   |   |─ QuestionCanvas.jsx
|   |   |   |─ TimerHUD.jsx
|   |   |   └─ AutoSaveIndicator.jsx
|   |   └─ Proctor/
|   |       |─ CameraPermissionGuard.jsx
|   |       |─ WebcamStream.jsx
|   |       |─ MediaPipeEngine.jsx
|   |       └─ WarningOverlay.jsx
|   └─ context/
|       |─ AuthContext.js
|       └─ ExamSessionContext.js # Global state for warnings, timers,
and connection
|   └─ hooks/
|       |─ useBrowserRestrictions.js
|       └─ useMediaPipeTasks.js
|   └─ pages/
|       |─ Login.jsx
|       |─ StudentDashboard.jsx
|       |─ AssessmentTerminal.jsx
|       |─ AdminConsole.jsx
|       └─ ExamAnalysisReport.jsx
|   └─ services/
|       |─ api.js
|       └─ socket.js # Singleton initialization for
Socket.IO-Client
|   └─ App.jsx
|   └─ index.js

```

5. Security & Browser Restriction System

Operating an assessment engine requires shutting down standard browser vectors that permit cheating. This module directly binds standard JavaScript events and blocks them.

JavaScript Security Interceptors Blueprint:

```

// useBrowserRestrictions.js (Custom React Hook)
import { useEffect } from 'react';

export const useBrowserRestrictions = (onTriggerViolation,
isExamActive) => {
  useEffect(() => {
    if (!isExamActive) return;

    // 1. Tab Switching & Window Minimization Detection
    const handleVisibilityChange = () => {

```

```

    if (document.hidden) {
        onTriggerViolation('TAB_SWITCHED', 'High');
    }
};

// 2. Fullscreen Exit Interception
const handleFullscreenChange = () => {
    if (!document.fullscreenElement) {
        onTriggerViolation('FULLSCREEN_EXIT', 'Medium');
    }
};

// 3. Disable Context Menus & Copy-Paste Operations
const blockContext = (e) => e.preventDefault();
const blockCopyPaste = (e) => {
    e.preventDefault();
    onTriggerViolation('COPY_PASTE_ATTEMPT', 'Low');
};

// 4. Keyboard Shortcut Blocking (F12, DevTools, Page Saving)
const blockKeyboardShortcuts = (e) => {
    const forbiddenKeys = ['F12'];
    const isDevToolsCombo = (e.ctrlKey && e.shiftKey && (e.key ===
'I' || e.key === 'J' || e.key === 'C'));
    const isPageSourceCombo = (e.ctrlKey && (e.key === 'U' || e.key
=== 'u'));
    const isCopyPasteCombo = (e.ctrlKey && (e.key === 'c' || e.key
=== 'v' || e.key === 'C' || e.key === 'V'));

    if (forbiddenKeys.includes(e.key) || isDevToolsCombo ||
isPageSourceCombo) {
        e.preventDefault();
        onTriggerViolation('DEVTOOLS_INTRUSION_ATTEMPT', 'Critical');
    }

    if (isCopyPasteCombo) {
        e.preventDefault();
        onTriggerViolation('COPY_PASTE_ATTEMPT', 'Low');
    }
};

// Add listeners
document.addEventListener('visibilitychange',
handleVisibilityChange);
document.addEventListener('fullscreenchange',
handleFullscreenChange);
document.addEventListener('contextmenu', blockContext);
document.addEventListener('copy', blockCopyPaste);

```

```

document.addEventListener('paste', blockCopyPaste);
document.addEventListener('cut', blockCopyPaste);
window.addEventListener('keydown', blockKeyboardShortcuts);

// Clean up on component unmount
return () => {
  document.removeEventListener('visibilitychange',
handleVisibilityChange);
  document.removeEventListener('fullscreenchange',
handleFullscreenChange);
  document.removeEventListener('contextmenu', blockContext);
  document.removeEventListener('copy', blockCopyPaste);
  document.removeEventListener('paste', blockCopyPaste);
  document.removeEventListener('cut', blockCopyPaste);
  window.removeEventListener('keydown', blockKeyboardShortcuts);
};
}, [isExamActive, onTriggerViolation]);
};

```

6. AI Proctoring Math & MediaPipe Setup

Google MediaPipe runs inside a web worker or a main thread RAF (RequestAnimationFrame) loop, mapping 478 points of the face mesh to find face density, head roll/pitch/yaw angles, and gaze vectors.

A. Core Mathematical Foundations

1. Real-Time Risk Score Calculations

The cumulative risk score R_{score} of a proctored session is determined as:

$$R_{\text{score}} = \min\left(100, \sum_{i=1}^n w_i \cdot c_i\right)$$

Where w_i represents the static weight assigned to violation type i , and c_i is the count of occurrences of violation i within the current session.

2. Head Orientation Estimation

The estimation of the face orientation matrix calculates yaw (θ) and pitch (ϕ) vectors. If we assume key landmarks of the human eye corner and nasal bridge correspond to a canonical coordinate frame:

$$\theta_{\text{yaw}} \approx \arctan2\left(x_{\text{nose}} - \frac{x_{\text{left_eye}} + x_{\text{right_eye}}}{2}, z_{\text{nose}} - z_{\text{eyes}}\right)$$

If $|\theta_{\text{yaw}}| > 35^\circ$, the system triggers a LOOKING_AWAY warning alert.

B. Client-Side Implementation Blueprint

This code utilizes Google MediaPipe's Face Landmarker API directly on incoming camera stream frames inside the React viewport.

```
import { FilesetResolver, FaceLandmarker } from
 '@mediapipe/tasks-vision';

let faceLandmarkerInstance = null;

// Initialize WebAssembly engine and load the deep neural networks
export const initializeFaceTracker = async () => {
  const vision = await FilesetResolver.forVisionTasks(

    "[https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@latest/wasm] (https://cdn.jsdelivr.net/npm/@mediapipe/tasks-vision@latest/wasm) "

  );

  faceLandmarkerInstance = await FaceLandmarker.createFromModelPath(
    vision,

    "[https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/float16/1/face_landmarker.task] (https://storage.googleapis.com/mediapipe-models/face_landmarker/face_landmarker/float16/1/face_landmarker.task) "

  );

  faceLandmarkerInstance.setOptions({
    runningMode: "VIDEO",
    numFaces: 2, // Setup to trigger multiple-face detection
    minFaceDetectionConfidence: 0.5,
    minFacePresenceConfidence: 0.5,
    outputFaceBlendshapes: true
  });

  return faceLandmarkerInstance;
};

// Process frames and evaluate violations
export const processWebcamFrame = (videoElement, timestamp) => {
  if (!faceLandmarkerInstance) return null;

  const results = faceLandmarkerInstance.detectForVideo(videoElement,
    timestamp);
  const violations = [];

  // 1. Monitor face counts
  const faceCount = results.faceLandmarks ?
    results.faceLandmarks.length : 0;
  if (faceCount === 0) {
    violations.push({ type: 'NO_FACE_DETECTED', severity: 'High' });
  }
}
```

```

    } else if (faceCount > 1) {
      violations.push({ type: 'MULTIPLE_FACES_DETECTED', severity:
'Critical' });
    } else {
      // 2. Perform Yaw (Yaw/Pitch angle tracking) using Landmark
positions
      const landmarks = results.faceLandmarks[0];
      const noseBridge = landmarks[4];      // Nasal tip
      const leftEyeOuter = landmarks[33];    // Outer left eye
      const rightEyeOuter = landmarks[263];  // Outer right eye

      // Simple Euclidean midpoints
      const eyeCenter = {
        x: (leftEyeOuter.x + rightEyeOuter.x) / 2,
        y: (leftEyeOuter.y + rightEyeOuter.y) / 2
      };

      const deltaX = noseBridge.x - eyeCenter.x;

      // Boundary Threshold Checks
      if (deltaX < -0.05) {
        violations.push({ type: 'LOOKING_AWAY_LEFT', severity: 'Medium'
});
      } else if (deltaX > 0.05) {
        violations.push({ type: 'LOOKING_AWAY_RIGHT', severity: 'Medium'
});
      }
    }

    return { faceCount, violations };
  };

```

7. Warning Matrix & Automated Penalties

Proctor violations execute instant UX alerts. The exam auto-submits once maximum thresholds are exceeded to lock down the session against repeated manipulation attempts.

Trigger Event	Severity Weight	Instant UX Response	Immediate Socket Emit	Target Penalty Threshold
Tab Switched	20 pts	Orange Banner Warning popup	Yes	Terminated if down for >20s
No Face Detected	15 pts	Red Banner Modal ("Face Invisible")	Yes	Warning Count +1 if >5s
Multiple Faces	40 pts	Immediate Red Fullscreen Shield	Yes	Instant Pause (requires verification)
Gaze Left / Right	10 pts	Yellow Notification	No	Adds warning


```

| <===== FORWARD: forcePauseExamEvent (payload)
===== |
|

```

Server-Side Socket.IO Event Handler Blueprint:

```

// sockets/proctorSocket.js
const jwt = require('jsonwebtoken');

module.exports = (io) => {
  io.use((socket, next) => {
    // Authenticate users joining the Socket Stream
    const token = socket.handshake.auth.token;
    if (!token) return next(new Error("Authentication Missing"));

    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
      if (err) return next(new Error("Authentication Corrupted"));
      socket.user = decoded;
      next();
    });
  });

  io.on('connection', (socket) => {
    console.log(`Live telemetry connection started: ${socket.id}
(User: ${socket.user.id})`);

    // Student joins specific exam room
    socket.on('joinExamSession', ({ examId, sessionId }) => {
      socket.join(`session_${sessionId}`);
      socket.join(`exam_${examId}`);
      console.log(`Examinee associated with Exam Room:
exam_${examId}`);
    });

    // Admins subscribe to live exam alerts
    socket.on('joinAdminLiveMonitor', ({ examId }) => {
      socket.join(`admin_monitor_${examId}`);
      console.log(`Invigilator registered for stream:
admin_monitor_${examId}`);
    });

    // Real-time telemetric logging pipeline
    socket.on('proctorTelemetryEvent', async (payload) => {
      const { sessionId, examId, eventType, riskDelta,
currentTimestamp } = payload;

    // Fast forward the events straight to listening monitors

```

```

without waiting for DB completion
    io.to(`admin_monitor_${examId}`).emit('liveAdminFeedBroadcast',
{
    studentId: socket.user.id,
    studentName: socket.user.name,
    eventType,
    riskDelta,
    timestamp: currentTimestamp
  });

  // Persist the event to MongoDB concurrently
  try {
    await mongoose.model('ProctorLog').create({
      sessionId,
      studentId: socket.user.id,
      examId,
      eventType,
      severity: riskDelta >= 30 ? 'high' : riskDelta >= 15 ?
'medium' : 'low',
      timestamp: new Date(currentTimestamp)
    });
  } catch (err) {
    console.error("Critical database error resolving proctor
telemetry logging: ", err);
  }
});

socket.on('disconnect', () => {
  console.log(`Socket disconnected: ${socket.id}`);
});
});
};

```

9. Comprehensive REST API Specifications

The system uses standard HTTP methods for administrative operations, data retrieval, and session state management.

Base Endpoint: <https://api.smaart.institute/api/v1>

Method	Endpoint	Auth	Request Payload	Response Object
POST	/auth/login	Public	{ "email": "str", "password": "str" }	{ "token": "jwt", "user": { "role": "str" } }
GET	/exams	Student/Admin	None	[{ "id": "id", "title": "str", "duration": 60

Method	Endpoint	Auth	Request Payload	Response Object
				}}
POST	/exams/initiate	Student	{ "examId": "mongold" }	{ "session": { "id": "mongold", "questions": [...] } }
PUT	/exams/autosave	Student	{ "sessionId": "id", "answers": [...] }	{ "success": true, "updatedAt": "Date" }
POST	/exams/terminate	Student/Admin	{ "sessionId": "id", "reason": "str" }	{ "status": "submitted", "finalRisk": 78 }
GET	/admin/sessions/live	Admin	None	[{ "student": "str", "risk": 42, "warnings": 2 }]
GET	/admin/logs/:sessionId	Admin	None	[{ "eventType": "str", "timestamp": "Date" }]

10. Estimated 3-Week Implementation Timeline

This plan is optimized for a modular rollout to ensure continuous functionality and testing.

Week 1: Core Engine, Shuffling, & Base Infrastructure (Focus: Functional MVP)

- **Milestones:** Complete basic question delivery API, design robust schemas, implement core exam screen routing.
- **Daily Tasks:**
 - *Day 1-2:* Database setup in MongoDB, auth, and question-bank generation routes. Implement question bank randomization (36 items selected from a bank of 350).
 - *Day 3-4:* Core Student Terminal UI layout featuring navigation tabs, auto-save state controllers, and reactive timers.
 - *Day 5:* Integrate backend auto-save endpoint to prevent progress loss. Verify and stress-test REST latency under simulated packet dropouts.

Week 2: Security Shell & Client-Side Machine Learning Integration (Focus: Proctor Security)

- **Milestones:** Implement operational browser lockdowns, launch the local WebAssembly MediaPipe pipeline, and configure the Client Warning Controller.
- **Daily Tasks:**
 - *Day 1-2:* Write and wire useBrowserRestrictions triggers. Test keyboard-shortcuts interceptors and verify multi-browser compliance (Chrome, Safari, Firefox).
 - *Day 3-4:* Build the WASM-loaded MediaPipeEngine controller component. Benchmark face boundary evaluations (478 landmarks) on varying hardware configurations (mobile, entry-level notebooks).
 - *Day 5:* Merge browser warning structures with the MediaPipe outputs. Connect

warnings with a progressive auto-submit trigger when constraints are repeatedly breached.

Week 3: Socket Integration, Live Invigilator Dashboards, and Reporting (Focus: Full Operational Scope)

- **Milestones:** Configure Socket.IO telemetry feeds, design real-time Admin Monitoring Grids, and build audit reporting sheets.
- **Daily Tasks:**
 - *Day 1-2:* Spin up server-side WebSocket namespaces. Implement connection token validation and emit telemetry events directly from client actions.
 - *Day 3:* Connect the admin real-time monitoring console to display live feeds, ongoing warning charts, and dynamic risk indicators.
 - *Day 4:* Implement full analytical summary reports for administrative audits post-exam, charting high-resolution event logs on custom timelines.
 - *Day 5:* Complete end-to-end sandbox QA runs. Finalize performance analysis and proceed to AWS/Vercel deployment setup.